How the FPL System Actually Works

An illustrated plain-language technical walkthrough

Written for a reader with a business/finance background. No programming assumed.

1 The bird's-eye view

Fantasy Premier League is, structurally, a portfolio management problem. Every week you hold a portfolio of 15 assets (players), each with a price, and each paying an uncertain weekly dividend (points). You have a fixed budget, position limits, diversification rules (max 3 players per real club), and transaction costs (transfers beyond the first cost you 4 points). Your job is to maximize total dividends over the season.

Any investment process like this splits into two distinct jobs, and so does this system:

1. **The forecaster** — the "research department." For every player, every week, it estimates how many points he will score in the next few gameweeks. This is a statistical/machine-learning problem.
2. **The optimizer** — the "portfolio construction desk." Given those forecasts, it works out the best legal squad, which transfers to make, whom to captain, and when to play chips. This is a mathematical optimization problem with a known, exact answer once the forecasts are fixed.

The separation matters because the two jobs fail differently. The forecaster's world is noisy and probabilistic — it can only be "less wrong." The optimizer's world is deterministic — given inputs, it finds the provably best squad, so if the final squads are bad, the fault is almost always the forecasts, not the optimization.

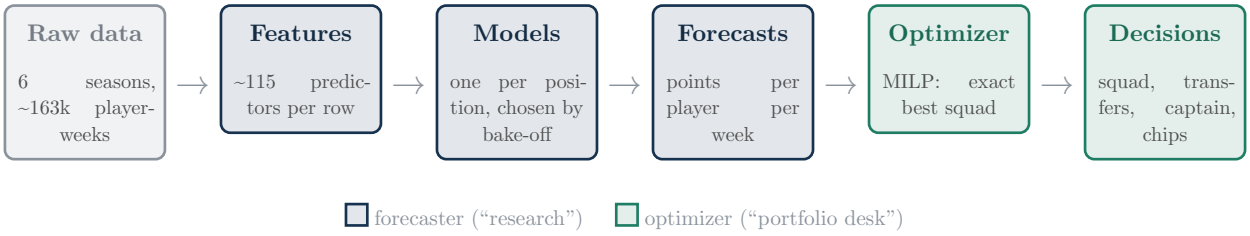

Figure 1: The pipeline. Everything left of the optimizer is statistics; everything from the optimizer on is exact mathematics.

2 The data

The raw material is one row per **player per gameweek**: minutes played, goals, assists, bonus points, expected goals (xG — a bookmaker-style estimate of how many goals a player "should" have scored given the quality of his shots), the opponent, home/away, price, ownership, and so on. Six seasons (2020-21 through 2025-26) give roughly **163,000 rows**.

Two sourcing details worth knowing:

- Historical data comes from a well-maintained public archive of FPL statistics; live data — next week's fixtures, your current squad — comes from FPL's official API, since the archive has no rows for matches not yet played.
- Gameweeks are numbered on one continuous counter across all seasons (GW 1-228 so far) rather than resetting each August. This makes "train on everything before week t" trivial to express — which, as you'll see, is the single most important discipline in the whole system.

3 Feature engineering: turning history into predictors

A model cannot eat a player's biography; it eats numbers. A **feature** is one number per player-week that summarizes something predictive. This system builds ~115 of them per row:

- **Form** — rolling averages of points, minutes, xG etc. over the last 3 and 5 games, plus a season-to-date and a career-to-date average. Like the 1-month, 3-month, YTD and since-inception returns on a fund factsheet: same quantity, different memory lengths.
- **Exponentially-weighted form** — recent games count more (half-life of 3 gameweeks). The finance analogue is exponentially-weighted volatility (RiskMetrics) versus a simple window.
- **Efficiency rates** — goals and xG per 90 minutes rather than per game, so a striker who scored in a 20-minute cameo isn't confused with one who needed the full match.
- **"Nailedness"** — the share of recent games started and played 60+ minutes. A brilliant player who doesn't get on the pitch pays no dividend.
- **Opponent strength** — the upcoming opponent's rolling attacking and defensive form, computed at team level; what separates "Haaland at home to the bottom club" from "Haaland away at Arsenal."
- **Fixture difficulty** — FPL's published 1-5 difficulty rating for the coming fixtures.
- **FPL's own forecast (xP)** — FPL publishes its own expected-points number pre-match; we feed in its recent history, like including consensus analyst estimates in your own equity model.

3.1 The one rule everything depends on: no peeking

Every feature derived from a player's own performance is **shifted back one gameweek**: the row for gameweek t contains only information from $t-1$ and earlier. The forecast for next Saturday cannot contain anything from next Saturday. This is the modeling equivalent of **look-ahead bias** in a trading backtest — a strategy that "knew" tomorrow's close looks brilliant in-sample and dies in production — and it creeps in through cracks, not the front door. Two real bugs found and fixed in this project's internal review:

- The live weekly forecast was accidentally reusing each player's **last played** match row, so its "fixture difficulty" described last week's opponent, not next week's.
- The model-blending weights (Section 4.5) were estimated on the very window the backtest was scored on — a subtle self-grading loop worth roughly **100 phantom points** (Section 7).

One more honesty rule: statistics that didn't exist in early seasons (xG wasn't collected before 2022-23) are stored as **missing**, not zero. "We didn't measure it" and "he generated exactly none" are different facts, and the models treat them differently.

4 The forecasting models

4.1 Why four models, not one

The system trains a separate model per position (GK, DEF, MID, FWD). What predicts a goalkeeper's points (saves, clean sheets) has almost nothing in common with what predicts a forward's (xG, penalty duty). Pooling them forces one set of rules to serve four different games.

4.2 The registry: twelve candidates and an index to beat

Rather than betting on one algorithm, the project maintains a **registry** of twelve — from plain linear regression up through modern gradient-boosted trees (LightGBM, XGBoost, CatBoost), random forests, nearest-neighbour and support-vector methods. Every candidate is trained and scored identically, and results are reported even when a method flops: the research log reads like a lab notebook, not a highlight reel.

One member has special status: **ordinary least squares (OLS) regression is the designated index**. Exactly like a passive benchmark in asset management, every fancier method must beat plain OLS on held-out data to justify its complexity. (Several don't.)

MASE, averaged across positions on held-out weeks — lower is better; below 1.0 beats the naive rule

Figure 2: What "forecast skill" looks like here. The naive "he'll score what he scored last week" rule defines 1.0; the index beats it comfortably; the production model beats the index. Fantasy points are so noisy that even the best model only removes about 30% of the naive error.

4.3 What a gradient-boosted tree is, in one paragraph

The workhorse models here are **gradient-boosted decision trees**. A decision tree is a flowchart of yes/no questions ("has he averaged over 4 points in the last 5 games? is the opponent's defensive form weak?") ending in a numeric prediction. One tree is crude. Boosting builds hundreds of small trees **in sequence, each fit to the errors the previous ones still make** — a chain of specialists, each correcting the last. For tabular data like ours, this family has dominated practical machine learning for a decade — which is why it displaced the neural network (LSTM) this project started with: sequence models need far more data per player than 38 gameweeks a season provides, while boosted trees pool learning across all players.

4.4 The median-versus-mean trap

Models are trained by minimizing a **loss function**, and that choice quietly decides **what kind of guess** the model makes: squared-error loss produces the conditional **mean**, absolute-error (MAE) loss produces the conditional **median**. For symmetric data those coincide. FPL points are wildly asymmetric — most weeks a player returns 2 points, occasionally he explodes for 15 — like income distributions, where a few enormous values pull the mean far above the median.

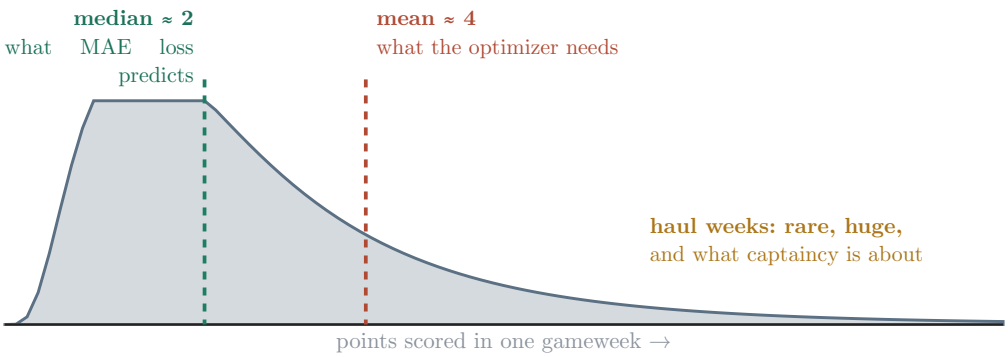

Figure 3: A right-skewed points distribution (illustrative). One model cannot serve three masters: accuracy metrics reward the median, squad value depends on the mean, captaincy depends on the tail.

Our best forecaster, CatBoost trained with MAE loss, predicts something close to each player's **median** week — its forecasts sum to only about half the points actually scored league-wide. Is that bad? It depends what the number is used for, and this became a central finding:

- For **ranking** players ("who is better this week?"), median-style forecasts are excellent — this model identifies the week's top scorer better than any alternative tested.
- For **absolute-scale decisions** (is a transfer worth its 4-point fee?), a forecast at half the true level distorts the exchange rate between points and fees.

We tested the obvious fix — rescaling each position's forecasts so they sum correctly, estimated on past data only — and it made the squads **worse** (Section 7). A theoretically motivated correction is a hypothesis, not a fix, until the backtest agrees.

4.5 Combining models: a diversification story that failed, instructively

With twelve forecasters, the natural instinct is to blend them — diversification for models. The system estimated optimal blend weights, and for a long time the blend was the production forecaster. Then a clean head-to-head showed the **single best model beat the twelve-model blend at every position**.

> **The Markowitz parallel**
>
> Optimized portfolio weights, estimated from finite noisy data, routinely lose out-of-sample to naive equal weighting — estimation error in the weights swamps the theoretical gain from optimizing. The same mathematics bites here (Clemen 1989, in the forecasting literature): twelve highly correlated forecasters, weights estimated on a short window — the weights were fitting noise. We tested the standard remedies too (equal-weighting the top 3, ridge-shrunk weights); the single best model still won.

So production is **one CatBoost model per position** — chosen not by ideology but by a **bake-off** re-run on every training pass: single-best vs. three combination schemes, all scored on the same held-out weeks. If a future change ever makes a blend win again, production follows the evidence automatically.

5 How we know whether any of this works

5.1 Walk-forward evaluation: the honest backtest

All evaluation mimics real life: to forecast gameweek t, train only on gameweeks strictly before t; predict; roll forward; repeat. No model, blend weight, scaling factor or tuning parameter is ever estimated on data from the period it is judged on — exactly how a quant fund should backtest a strategy, for exactly the same reason.

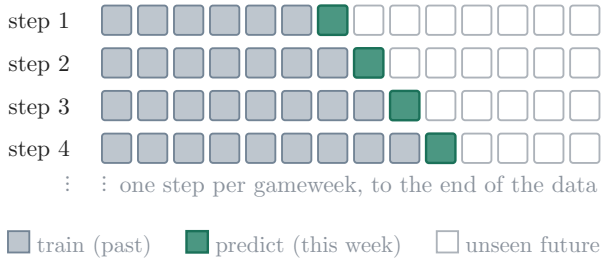

Figure 4: Walk-forward validation: the training window only ever grows into the past; the week being predicted is never part of it. Each square is a gameweek.

5.2 The scorecards

- **MAE** — on average, how many points off is each player-week forecast? (~0.85-1.05 here.)
- **MASE** — MAE divided by the error of a naive "same as last week" forecast; below 1.0 beats naive. The headline accuracy metric, because raw MAE is uninterpretable for a bursty series.
- **Decision-aligned diagnostics** — calibration (do forecasts sum to points actually scored?), within-week rank correlation, and **top-1 capture** (how often is our predicted best player actually the week's best — the captaincy question). These exist because of Section 4.4: a model can improve MASE while becoming worse for decisions.
- **The gold standard: realized points** — feed the forecasts through the optimizer over a full historical season and count what the squads actually scored. The only number that measures the whole system.

That last one earns its title because — demonstrated twice in this project — **forecast accuracy improvements do not reliably produce better squads**. A feature upgrade once improved MASE at every position while realized points dropped 1,966 → 1,880. Accuracy metrics are the compass; realized points are the terrain.

6 The optimizer

Given predicted points for every player over the next few weeks, squad selection becomes a **mixed-integer linear program** (MILP) — "integer" because you cannot hold 0.4 of a defender. It is the same mathematical species as portfolio optimization with lot-size constraints, and it is solved **exactly**: maximize predicted points subject to budget (£100m), squad shape (2 GK / 5 DEF / 5 MID / 3 FWD), max 3 per club, a legal starting XI, captain scoring double, transfer accounting (one free per week, extras cost 4 points), and one-time chips.

Because optimizing a whole season at once is computationally hopeless (and pointless — forecasts degrade with distance), it solves on a **rolling horizon**: optimize weeks t to $t+2$, commit only week t's decisions, roll forward, re-solve. Corporate planning under uncertainty works the same way. An

automated test suite checks every output against all the rules above, so a code change that quietly produces illegal squads is caught by tests, not by a mysterious backtest number.

7 What the numbers actually say

All figures are realized points over the same 31-gameweek window (2024-25 season, weeks 1-31), same optimizer, directly comparable:

*Figure 5: The full lineage. Honest validation first **lowered** the headline (the amber bar was partly phantom), then hyperparameter tuning delivered the largest genuine gain in the project's history.*

Three takeaways worth internalizing, because they generalize far beyond fantasy football:

1. **Most of the improvement came from process, not cleverness** — honest validation, leakage fixes, longer history, automated tuning, and letting a bake-off (not a preference) choose the model. The glamorous additions (37 new features in one batch; model blending; level recalibration) added roughly nothing or less than nothing. The single largest gain — tuning, +251 points — came from patiently searching configuration knobs, and its winning settings tell their own story: slower learning rates and shallower trees than the hand-set defaults, i.e. the defaults were overconfident.
2. **The metric you optimize is a design decision with consequences.** MAE-trained models predict medians; optimizers consume absolute scales; captaincy consumes tail behaviour (Figure 3). No single number captures all three, which is why the system reports a dashboard, not one score.
3. **Negative results are assets.** The failed recalibration, the failed blend, the accuracy-up/points-down episodes — each is logged with numbers and mechanism in the research log, and each permanently changed how the next experiment is judged.

8 Where the project goes from here

In priority order — all subject to the same rule: beat 2,107 realized points honestly, or be reported as a negative result.

- **Consolidate the tuning win**: retrain production on the tuned parameters, tune the other boosted models so the registry comparison is fair (a tuned blend deserves a re-match), and stability-check the 2,107 on a second window and seed.

- **Upside-aware captaincy**: a parallel quantile model already estimates each player's 90th-percentile week; feeding that into the captain choice attacks the tail directly.
- **Component decomposition** (the big architectural bet): predict points' ingredients — minutes, goals, assists, clean sheets — and recombine. Clean sheets are a team-level event, so eleven defenders' forecasts could share one team model.
- **A live-season interface**: the weekly driver already refreshes data, retrains, and prints recommended transfers via the FPL API; a friendlier surface is future work once the 2026-27 season opens.

Companion documents: `report/main.typ` *(formal write-up, full methodology)* · `RESEARCH_LOG.md` *(chronological lab notebook, including everything that failed)* · `TODO.md` *(open items)* · `CLAUDE.md` *(architecture reference).*